Tusdatos.co

Prueba técnica Desarrollador Backend

La empresa organizadora de eventos *Mis Eventos* gestiona sus eventos de forma tradicional y manual con lo que conlleva una serie de problemas listados a continuación:

● **Desorganización:** Falta de una plataforma centralizada para almacenar y gestionar toda la información relacionada con el evento (inscripciones, horarios, recursos, etc.).
● **Procesos manuales:** Muchas tareas se realizan de forma manual, lo que consume tiempo y es propenso a errores (e.g., registro de asistentes, generación de reportes).
● **Dificultad para tomar decisiones:** Falta de datos precisos y análisis para tomar decisiones estratégicas sobre los eventos.
● **Mala experiencia del usuario:** Los asistentes encuentran dificultades para encontrar información y realizar inscripciones, lo que afecta la satisfacción del cliente.
● **Organización y eficiencia:** Centraliza la información, automatiza procesos y reduce errores.

Objetivo

Desarrollar una aplicación web Full Stack que permita a "*Mis Eventos*" administrar sus eventos de manera eficiente, desde la creación hasta el registro de asistentes.

Funcionalidades requeridas

Backend (FastAPI o Flask, PostgreSQL o MongoDB)

1. Gestión de eventos con búsqueda avanzada
    - CRUD completo de eventos
    - Gestión de capacidad y registro
    - Estados del evento
    - Validaciones de negocio
    - Búsqueda de eventos por nombre o parte del nombre.
2. Registro y autenticación de usuarios:

    - Registro de usuarios (email, contraseña).
    - Login de usuarios.
    - Protección de rutas (solo usuarios autenticados pueden crear/editar eventos).
3. Programación de sesiones
    - Creación y gestión de sesiones
    - Asignación de ponentes
    - Validación de horarios
    - Control de capacidad
4. Gestión de asistentes:
    - Permitir al usuario autenticado registrarse a un evento (si hay capacidad).
    - Permitir al usuario autenticado ver los eventos a los que está registrado.
    - Controlar la capacidad de los eventos.

Bonus (Opcional):

- Roles de usuarios (Admin, Organizador, Asistente)
- Permisos de acceso por rol
- Gestión de perfiles por parte del administrador.

Frontend (React, Vue.js o Angular):

1. Interfaz principal:
    - Listado de eventos disponibles (con paginación).
    - Detalle de un evento (mostrando información y sesiones).
    - Formulario de creación/edición de eventos (solo para usuarios autenticados).
    - Formulario de registro/login de usuarios.
2. Navegación (Routing):
    - Rutas claras para: lista de eventos, detalle de evento, crear evento, login, registro, perfil de usuario (donde se ven los eventos a los que está registrado).
3. Gestión de estados:
    - Utilizar un manejador de estado.
    - Gestionar el estado de autenticación (usuario logueado, token).
    - Gestionar el estado de los eventos (lista, evento actual).
4. Interacción con el Backend:

    - Consumir los endpoints del API REST del backend para todas las operaciones (CRUD de eventos, autenticación, registro a eventos).
    - Manejo adecuado de errores (mostrar mensajes al usuario).
5. Componentes reutilizables:
    - Crear componentes reutilizables (e.g., EventoCard, Input, Button, FormularioEvento).
    - Buena estructura de componentes y props.

Bonus (Opcional):

- Optimización de Imágenes: Usar formatos modernos (WebP), compresión, diferentes tamaños para distintos dispositivos.
- Lazy Loading: Cargar componentes y rutas de forma diferida para mejorar el tiempo de carga inicial.
- Minificación de Código: Minificar y ofuscar el código de producción.
- Caching: Implementar caching de datos (en el frontend o usando Redis en el backend).
- Testing, linting
- UI/UX:
    - Implementar un diseño atractivo y usable.
    - Considerar la responsividad (adaptación a diferentes tamaños de pantalla).
    - Implementar loading states o spinners.
    - Implementar feedback visual para las acciones del usuario.

Requerimientos técnicos

1. Backend:
    - Python 3.12
    - FastAPI o Flask
    - SQLAlchemy + SQLModel (o SQLAlchemy Core)
    - PostgreSQL
    - Poetry (gestión de dependencias)
    - Alembic (migraciones)
    - Tests unitarios (pytest)
    - Documentación API (Swagger/OpenAPI)

2. Frontend:
    - React, Vue.js o Angular
    - Router de acuerdo al framework seleccionado
    - Manejador de estados
    - Manejo de peticiones HTTP
    - Tests unitarios
3. Entorno:
    - Docker y Docker Compose (para backend, frontend y base de datos)
    - Variables de entorno

Entregables

1. Código:
    - Repositorio Git con el código del backend y del frontend (en carpetas separadas).
    - READMEs separados para backend y frontend, explicando cómo configurar, ejecutar y probar cada parte.
2. Documentación:
    - Documentación API (Swagger/OpenAPI).
    - Documentación de la estructura del frontend (componentes, rutas, etc).
3. Despliegue:
    - Dockerfile para el backend.
    - Dockerfile para el frontend.
    - Docker Compose para orquestar todos los servicios (backend, frontend, base de datos).
4. Tests:
    - Tests unitarios para backend y frontend.
    - Reporte de cobertura de tests.

Plazo de entrega: Tienes 5 días para completar esta prueba desde la recepción de la misma. Envíala al correo de talento@tusdatos.co en el asunto indica cual es el nombre de la vacante a la cual te estás postulando.